

Festival

Nayra je na festivalu i igra igru u kojoj je glavna nagrada putovanje u Lagunu Coloradu. Igra se sastoji od korištenja žetona za kupovinu kupona. Kupovinom kupona možete osvojiti dodatne žetone. Cilj je dobiti što više kupona.

Ona započinje igru sa žetonima A . Postoje kuponi u vrijednosti N , numerisani od 0 do $N - 1$. Nayra mora platiti $P[i]$ tokena ($0 \leq i < N$) da bi kupila kupon i (i mora imati najmanje $P[i]$ tokena prije kupovine). Ona može kupiti svaki kupon najviše jednom.

Štaviše, svakom kuponu i ($0 \leq i < N$) je dodijeljen **tip**, označeno sa $T[i]$, što je cijeli broj **između 1 i 4, uključujući i te brojeve**. Nakon što Nayra kupi kupon i , Preostali broj žetona koje ima množi se sa $T[i]$. Formalno, ako ona u nekom trenutku igre ima X žetona, i kupuje kupon i (što zahtijeva $X \geq P[i]$), onda će nakon kupovine imati $(X - P[i]) \cdot T[i]$ tokena.

Vaš zadatak je da odredite koje kupone Nayra treba kupiti i kojim redoslijedom, kako bi maksimizirala ukupan broj **kupona** koje ima na kraju. Ako postoji više od jednog niza kupovina kojima se ovo postiže, možete prijaviti bilo koju od njih.

Detalji implementacije

Trebali biste implementirati sljedeći postupak.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
std::vector<int> T)
```

- A : početni broj tokena koje Nayra ima.
- P : niz dužine N koji specificira cijene kupona.
- T : niz dužine N koji specificira tipove kupona.
- Ova procedura se poziva tačno jednom za svaki testni slučaj.

Procedura treba da vrati niz R , koji specificira Nayrine kupovine na sljedeći način:

- Dužina R treba da bude maksimalan broj kupona koje ona može kupiti.
- Elementi niza su indeksi kupona koje treba kupiti, hronološkim redom. To jest, ona prvo kupuje kupon $R[0]$, zatim kupon $R[1]$ i tako dalje.
- Svi elementi R moraju biti jedinstveni.

Ako se ne mogu kupiti kuponi, R treba biti prazan niz.

Ograničenja

$$1 \leq N \leq 200\,000$$

- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ za svako i takvo da $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ za svako i takvo da $0 \leq i < N$.

Podzadaci

Podzadatak	Rezultat	Dodatna ograničenja
1	5	$T[i] = 1$ za svako i takvo da je $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ za svako i takvo da je $0 \leq i < N$.
3	12	$T[i] \leq 2$ za svako i takvo da je $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra može kupiti sve kupone u vrijednosti N (nekim redoslijedom).
6	16	$(A - P[i]) \cdot T[i] < A$ za svako i takvo da je $0 \leq i < N$.
7	18	Nema dodatnih ograničenja.

Primjeri

Primjer 1

Razmotrite sljedeći poziv.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra inicijalno ima tokene u $A = 13$. Ona može kupiti kupone 3 redoslijedom prikazanim ispod:

Kupljeni kupon	Cijena kupona	Vrsta kupona	Broj tokena nakon kupovine
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

U ovom primjeru, Nayra ne može kupiti kupone u vrijednosti većoj od 3. i redoslijed kupovina opisan gore je jedini način na koji ona može kupiti 3 od njih. Dakle, procedura bi trebala vratiti $[2, 3, 0]$.

Primjer 2

Razmotrite sljedeći poziv.

```
max_coupons(9, [6, 5], [2, 3])
```

U ovom primjeru, Nayra može kupiti oba kupona bilo kojim redoslijedom. Dakle, procedura bi trebala vratiti ili $[0, 1]$ ili $[1, 0]$.

Primjer 3

Razmotrite sljedeći poziv.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

U ovom primjeru, Nayra ima jedan token, što nije dovoljno za kupovinu bilo kojeg kupona. Dakle, procedura bi trebala vratiti $[]$ (prazan niz).

Primjer ocjenjivača

Format unosa:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Izlazni format:

```
S
R[0] R[1] ... R[S-1]
```

Ovdje je S dužina niza R kojeg vraća `max_coupons`.